Universidad Don Bosco Dirección de Educación a Distancia



Desarrollo de Software para Móviles DSM941

Docente: Alexande Alberto Sigüenza

Estudiantes:

Christian Geovanni Centeno, CS241743

José Alexander Montoya, MQ252529

Gabriel Quintanilla Rodríguez, QR230082

Actividad:

“Avance de Etapa 3: Foro 2”

Introducción **3**

Objetivos **3**

Objetivo general 3

Objetivos específicos 3

Marco teórico **4**

Firebase como plataforma de desarrollo 4

Firebase Authentication 4

Autenticación por correo electrónico y contraseña 5

Autenticación con Google 5

Custom claims 5

Cloud Firestore 5

Consultas y actualizaciones en tiempo real 6

Transacciones 6

Reglas de seguridad de Firestore 6

Firebase Local Emulator Suite 6

Aplicación en Panadería Rosario **7**

Arquitectura de la integración 7

Autenticación de usuarios 7

Rol de administrador 8

Modelo de datos en Firestore 8

Confirmación de pedidos con transacciones 8

Historial y actualizaciones en tiempo real 9

Reglas de seguridad del proyecto 9

Interfaz de la aplicación 10

Correspondencia con los requerimientos del avance 15

Ventajas y limitaciones **15**

Conclusiones **15**

Referencias **17**

Firebase Authentication y Cloud Firestore aplicados a la aplicación móvil Panadería Rosario

Introducción

Panadería Rosario es una aplicación Android desarrollada en Kotlin con Jetpack Compose que permite a los clientes consultar el catálogo, armar un carrito, confirmar pedidos y seguir su estado, mientras que el personal administrativo gestiona productos, existencias, entregas y reportes. En la etapa anterior, los datos residían únicamente en memoria y en el dispositivo, por lo que cada instalación tenía su propia información y no existía una identidad verificable para los usuarios.

En este segundo avance se integraron dos servicios de Firebase, la plataforma de desarrollo de aplicaciones de Google: Firebase Authentication, para registrar e identificar usuarios mediante correo electrónico o cuenta de Google, y Cloud Firestore, una base de datos NoSQL en la nube donde se almacenan perfiles, catálogo e historial de pedidos (Google, s. f.-g; Google, s. f.-d). El presente documento describe los fundamentos de ambos servicios, explica las reglas de seguridad que protegen la información y detalla cómo se aplicaron en el código del proyecto.

Objetivos

Objetivo general

Investigar los servicios Firebase Authentication y Cloud Firestore y documentar su integración en la aplicación Panadería Rosario como mecanismo de autenticación y persistencia remota de datos.

Objetivos específicos

- Describir el funcionamiento de Firebase Authentication con los proveedores de correo electrónico y Google.
- Explicar el modelo de datos, las consultas en tiempo real y las transacciones de Cloud Firestore.

- Analizar las reglas de seguridad y los custom claims como mecanismos de control de acceso.
- Relacionar cada concepto con los componentes implementados en el proyecto.
- Identificar ventajas y limitaciones de la solución adoptada.

Marco teórico

Firebase como plataforma de desarrollo

Firebase es un conjunto de servicios administrados que ofrece a las aplicaciones móviles y web funciones de backend, como autenticación, bases de datos, almacenamiento y análisis, sin que el equipo de desarrollo deba instalar ni mantener servidores propios. Este enfoque se conoce como *Backend as a Service* (BaaS). Moroney (2017) señala que su principal aporte es permitir que el desarrollador se concentre en la experiencia de la aplicación, delegando en la plataforma tareas como la escalabilidad, la sincronización y la seguridad de la infraestructura.

Para conectar una aplicación Android con un proyecto de Firebase, se registra la aplicación con su identificador de paquete y se descarga el archivo google-services.json, que contiene los datos de configuración del cliente. El complemento de Gradle com.google.gms.google-services procesa ese archivo y genera los recursos que utilizan las bibliotecas de Firebase (Google, s. f.-b). Este archivo no es una clave privada; la protección de los datos depende de la autenticación y de las reglas de seguridad.

Firebase Authentication

Firebase Authentication proporciona servicios de backend y bibliotecas cliente para identificar a los usuarios de una aplicación. Admite contraseñas, números de teléfono y proveedores federados como Google, Facebook o GitHub (Google, s. f.-g). Cada usuario recibe un identificador único llamado UID, que se mantiene constante independientemente del proveedor utilizado y que sirve para asociar sus datos en otros servicios.

Autenticación por correo electrónico y contraseña

Con este proveedor, la aplicación envía el correo y la contraseña al servicio, que se encarga de almacenarla de forma segura y de validar los intentos de inicio de sesión. La aplicación nunca guarda contraseñas propias. Tras un acceso exitoso, el SDK conserva la sesión en el dispositivo y emite un token de identidad (ID token) firmado, que se renueva automáticamente y acompaña a cada solicitud a otros servicios de Firebase (Google, s. f.-g).

Autenticación con Google

El acceso con Google en Android se realiza actualmente mediante Credential Manager, la API de Android que unifica contraseñas, llaves de acceso e inicio de sesión federado (Android Developers, s. f.). La aplicación solicita una credencial de Google indicando el ID de cliente OAuth de tipo web del proyecto; el usuario elige una cuenta y se obtiene un token de Google, que luego se intercambia por una credencial de Firebase mediante GoogleAuthProvider (Google, s. f.-c). Para que funcione, el proveedor debe habilitarse en la consola y deben registrarse las huellas SHA-1 y SHA-256 del certificado con que se firma la aplicación.

Custom claims

Los custom claims son atributos adicionales que se agregan al token de un usuario desde un entorno de servidor de confianza mediante el Firebase Admin SDK. Se utilizan para representar roles, como administrador, que luego pueden consultarse tanto en el cliente como en las reglas de seguridad. Como solo pueden asignarse con credenciales administrativas, un usuario no puede otorgárselos a sí mismo (Google, s. f.-f).

Cloud Firestore

Cloud Firestore es una base de datos NoSQL orientada a documentos, flexible y escalable, con sincronización en tiempo real y soporte sin conexión (Google, s. f.-d). A diferencia de una base relacional, no utiliza tablas ni filas: la información se organiza en documentos, que contienen pares campo-valor, y los documentos se agrupan en colecciones. Un documento puede contener mapas y listas anidadas, y se identifica por una ruta como pedidos/abc123 (Google, s. f.-e).

Consultas y actualizaciones en tiempo real

Firestore permite filtrar documentos con consultas como whereEqualTo y, además de lecturas puntuales, registrar *listeners* con addSnapshotListener. Un listener recibe una instantánea inicial y luego una nueva cada vez que los documentos que coinciden con la consulta cambian, sin que la aplicación tenga que volver a consultar (Google, s. f.-h). El SDK también guarda en caché los datos leídos, de modo que la aplicación puede mostrarlos y registrar escrituras aun sin conexión, sincronizándolas al recuperar la red (Google, s. f.-a).

Transacciones

Una transacción agrupa lecturas y escrituras que deben ejecutarse de forma atómica: o se aplican todas, o ninguna. Si otro cliente modifica alguno de los documentos leídos antes de confirmar, Firestore vuelve a ejecutar la función de la transacción con los datos actualizados (Google, s. f.-k). Este mecanismo es adecuado para operaciones como descontar existencias o asignar números consecutivos, donde varios usuarios pueden actuar al mismo tiempo.

Reglas de seguridad de Firestore

Como las aplicaciones cliente acceden directamente a la base de datos, Firestore utiliza reglas de seguridad declarativas que el servidor evalúa en cada lectura y escritura. Las reglas se escriben por ruta de documento y pueden consultar request.auth, que contiene el UID y los claims del usuario autenticado; resource.data, con los datos existentes; y request.resource.data, con los datos que se intentan escribir. Las funciones get, exists y getAfter permiten validar otros documentos involucrados en la misma operación (Google, s. f.-i). Si ninguna regla permite una operación, esta se rechaza.

Firebase Local Emulator Suite

Local Emulator Suite ejecuta versiones locales de Authentication, Firestore y otros servicios, lo que permite desarrollar y probar sin afectar datos reales ni generar costos. Las aplicaciones se conectan a los emuladores llamando a useEmulator con la dirección y el puerto correspondientes (Google, s. f.-j).

Aplicación en Panadería Rosario

Arquitectura de la integración

La integración se concentra en la clase FirebaseRepository, ubicada en el paquete data, que encapsula todas las llamadas a Authentication y Firestore mediante corrutinas de Kotlin y la función await de kotlinx-coroutines-play-services. La clase AppState, implementada como AndroidViewModel, coordina la sesión, el carrito y los listeners, y expone el estado a las pantallas de Jetpack Compose. Las clases de dominio de los paquetes model y service (Producto, Pedido, DetallePedido, PedidoService, entre otras) se conservaron de las etapas anteriores, por lo que Firebase se incorporó como capa de persistencia sin reescribir la lógica de negocio.

Las dependencias se gestionan con la Firebase BoM 33.7.0, que fija versiones compatibles de firebase-auth y firebase-firestore. El archivo de compilación aplica el complemento de Google Services solo cuando existe app/google-services.json. Si el archivo no está presente, la aplicación inicia en un modo de demostración local, identificado en pantalla, que guarda los datos en el almacenamiento privado del dispositivo. Esto permite que cualquier integrante compile y pruebe el proyecto antes de configurar el proyecto remoto.

Autenticación de usuarios

La pantalla de inicio de sesión ofrece tres opciones: ingresar con correo y contraseña, crear una cuenta y continuar con Google. El registro llama a createUserWithEmailAndPassword y, a continuación, guarda el perfil del cliente (nombre, teléfono, dirección y correo) en el documento usuarios/{uid}. El acceso con Google utiliza GetSignInWithGoogleOption de Credential Manager con el recurso default_web_client_id generado desde google-services.json, y entrega el token obtenido a signInWithCredential.

```
suspend fun google(idToken: String) {
    auth.signInWithCredential(
        GoogleAuthProvider.getCredential(idToken, null)).await()
}
```

Firebase mantiene la sesión entre reinicios. Al abrir la aplicación, recuperarSesion comprueba auth.currentUser y, si existe un usuario, restablece los listeners sin pedir de

nuevo las credenciales. Al cerrar sesión, los listeners se eliminan y el historial local se vacía para no mostrar datos de otra cuenta en el mismo dispositivo.

Rol de administrador

El acceso administrativo no depende de contraseñas fijas en el código ni de un campo editable en Firestore, sino del custom claim admin. El script scripts/set_admin.py se ejecuta en un entorno de confianza con el Admin SDK y agrega admin: true al usuario indicado. En la aplicación, esAdmin fuerza la renovación del token con getIdToken(true) y lee el claim; si la cuenta no lo tiene, el acceso administrativo se rechaza. Las reglas de seguridad verifican el mismo claim, por lo que la protección no depende solo de la interfaz.

Modelo de datos en Firestore

La información se organiza en cuatro colecciones, resumidas en la Tabla 1. Cada pedido almacena el UID de su propietario en el campo usuarioId, lo que cumple el requisito de asociar los datos al usuario autenticado.

Tabla 1

Colecciones de Firestore utilizadas por la aplicación

Ruta	Contenido	Acceso permitido
usuarios/{uid}	Nombre, teléfono, dirección y correo del cliente	El propietario; lectura del administrador
productos/{id}	Nombre, categoría, precio y existencias	Lectura de usuarios autenticados; edición del administrador
pedidos/{id}	UID, cliente, detalles, cantidades, total, estado y fecha del servidor	El propietario y el administrador
contadores/pedidos	Último número de pedido asignado	Actualización dentro de la transacción de un pedido

Nota. Elaboración propia a partir de firestore.rules y FirebaseRepository.kt.

Confirmación de pedidos con transacciones

La creación de un pedido se implementa con runTransaction. Dentro de la transacción se lee el contador de pedidos y cada producto del carrito, se verifica que el producto exista, que el precio no haya cambiado y que haya existencias suficientes; luego se incrementa

el contador, se descuentan las existencias y se crea el documento del pedido con estado RECIBIDO y la marca de tiempo del servidor (FieldValue.serverTimestamp()). Si dos clientes compran el último producto al mismo tiempo, solo una transacción se confirma y la otra recibe el error de stock insuficiente, mientras el carrito se conserva para reintentar.

```
val stock = doc.getLong("stock") ?: 0
require(stock >= detalle.cantidad) {
   "Stock insuficiente para ${detalle.producto.nombre}." }
tx.set(contador, mapOf("valor" to nuevo, "ultimoPedido" to ref.id))
```

Historial y actualizaciones en tiempo real

El historial de cada cliente se obtiene con un listener sobre la consulta pedidos filtrada por usuarioId igual al UID del usuario actual; el administrador observa la colección completa. Cuando el administrador avanza un pedido por los estados RECIBIDO, EN_PREPARACION, EN_CAMINO y ENTREGADO, el cambio se refleja de inmediato en la pantalla Mis pedidos del cliente. De igual forma, un listener sobre productos mantiene el catálogo actualizado en todos los dispositivos cuando el administrador agrega, edita o elimina productos.

Reglas de seguridad del proyecto

El archivo firestore.rules define funciones auxiliares signedIn() y admin() y aplica restricciones por colección. Un perfil solo puede escribirlo su propietario, con los campos permitidos y con el mismo correo de su token. Un pedido solo puede leerlo su propietario o un administrador, y solo puede crearse con estado RECIBIDO, la fecha del servidor y el UID del solicitante. Los cambios de estado están limitados al administrador y deben seguir el orden establecido:

```
allow update: if admin()
  && request.resource.data.diff(resource.data)
       .affectedKeys().hasOnly(['estado'])
  && ((resource.data.estado == 'RECIBIDO'
       && request.resource.data.estado == 'EN_PREPARACION') || ...);
```

Además, mediante getAfter, las reglas comprueban que el descuento de existencias de un cliente corresponda exactamente a las cantidades del pedido que se crea en la misma transacción, lo que impide reducir el stock de forma arbitraria. Las reglas se despliegan con Firebase CLI usando firebase.json, que también configura los emuladores de Authentication (puerto 9099) y Firestore (puerto 8080) para pruebas locales.

Interfaz de la aplicación

Las siguientes figuras muestran las pantallas principales de la aplicación. La Figura 1 corresponde a la compilación configurada con Firebase, donde se muestran el acceso con correo, el acceso con Google y el registro. Las demás pantallas se capturaron en el modo de demostración local, identificado por la leyenda superior, ya que comparten la misma interfaz que el modo conectado; en ese modo, los datos se guardan en el dispositivo en lugar de Firestore.

Figura 1

Pantallas de inicio de sesión y registro con Firebase Authentication

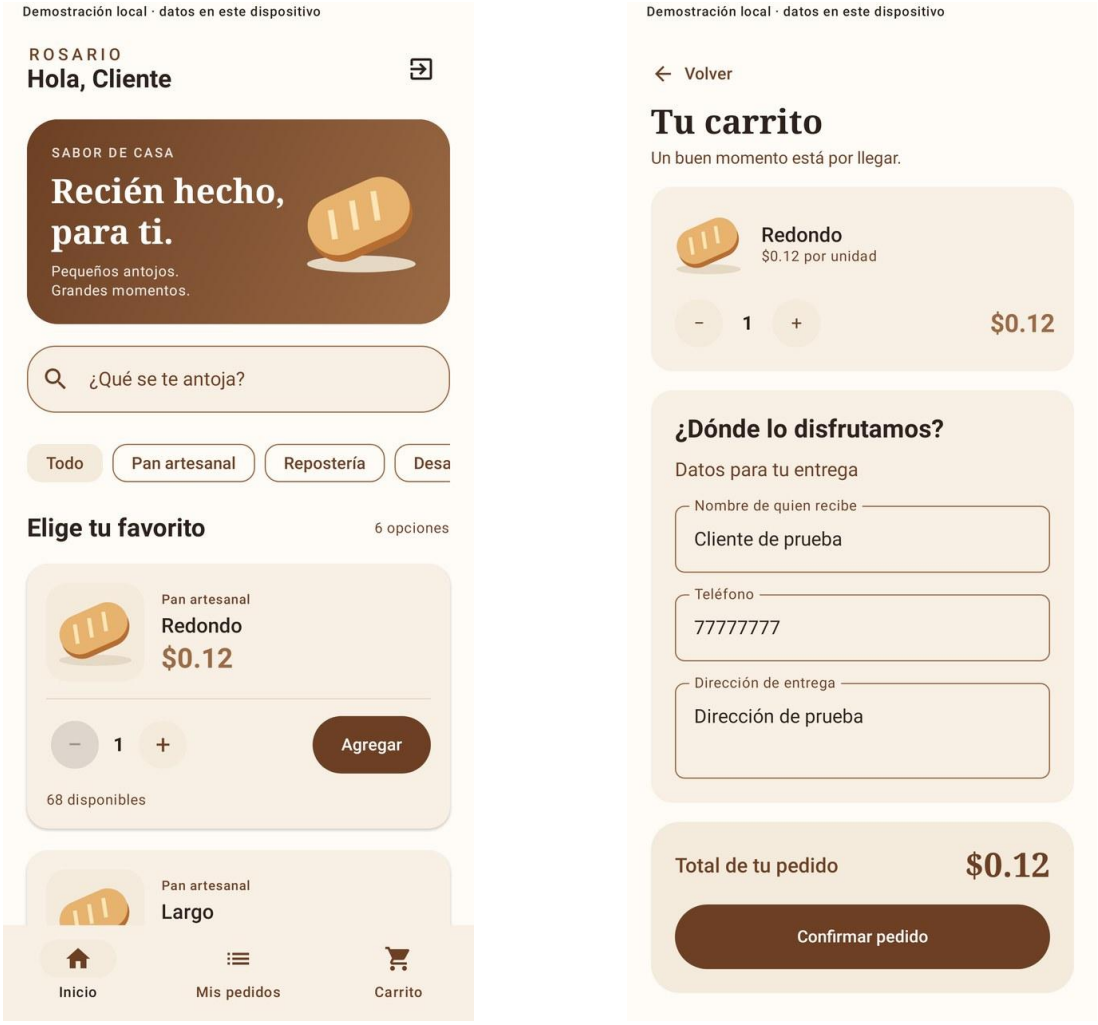


a) Inicio de sesión *b) Registro de cliente*

Nota. Captura de la compilación con Firebase configurado. El botón Continuar con Google utiliza Credential Manager; el registro crea la cuenta en Authentication y el perfil en usuarios/{uid}. Elaboración propia.

Figura 2

Catálogo y carrito del cliente

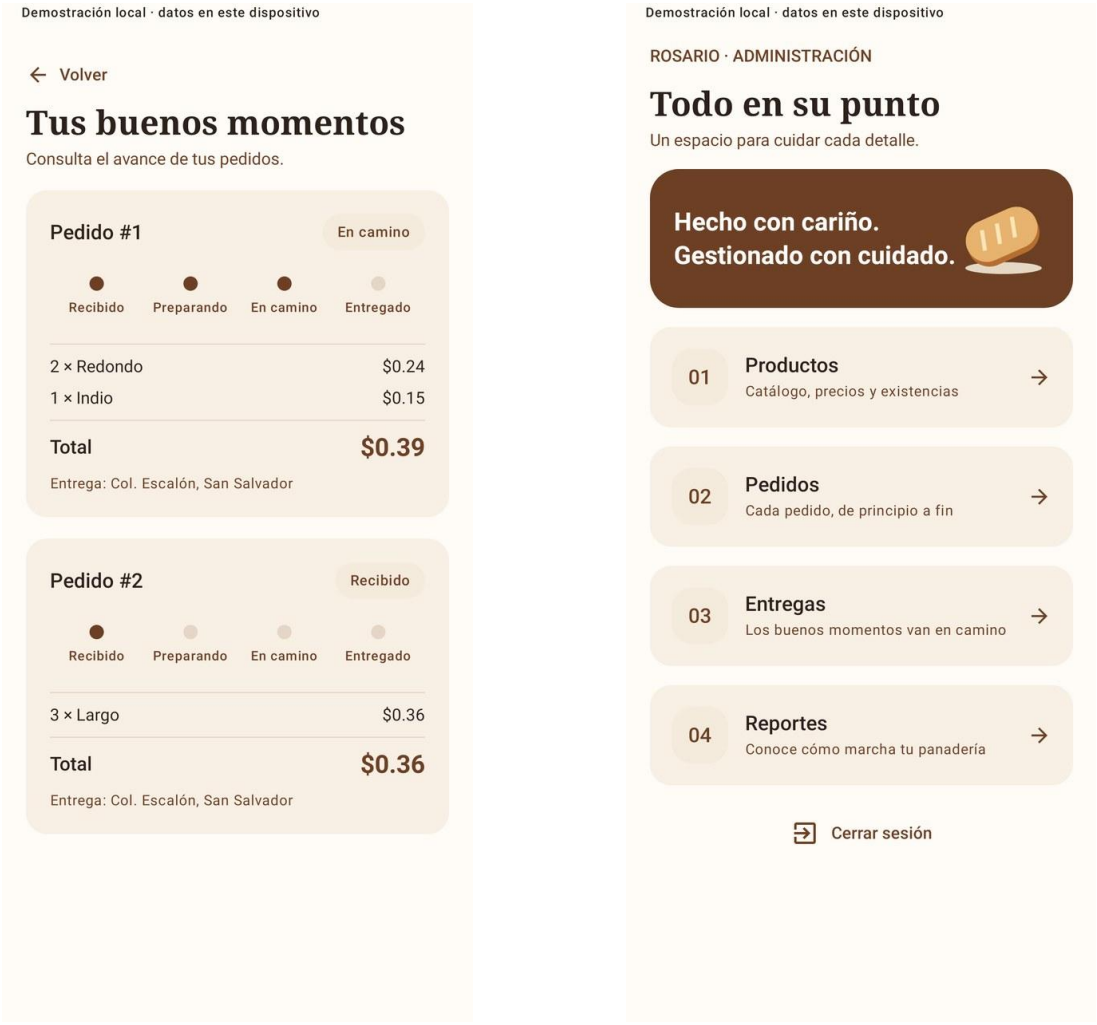


a) Catálogo con búsqueda y filtros *b) Carrito y datos de entrega*

Nota. Captura de la interfaz de la aplicación en modo de demostración local, que utiliza las mismas pantallas que el modo conectado a Firebase. Elaboración propia.

Figura 3

Historial de pedidos del cliente y panel de administración



a) Mis pedidos con seguimiento *b) Panel administrativo*

Nota. Captura de la interfaz de la aplicación en modo de demostración local, que utiliza las mismas pantallas que el modo conectado a Firebase. Elaboración propia.

Figura 4

Gestión de productos y de pedidos



a) Catálogo y existencias



b) Actualización de estados

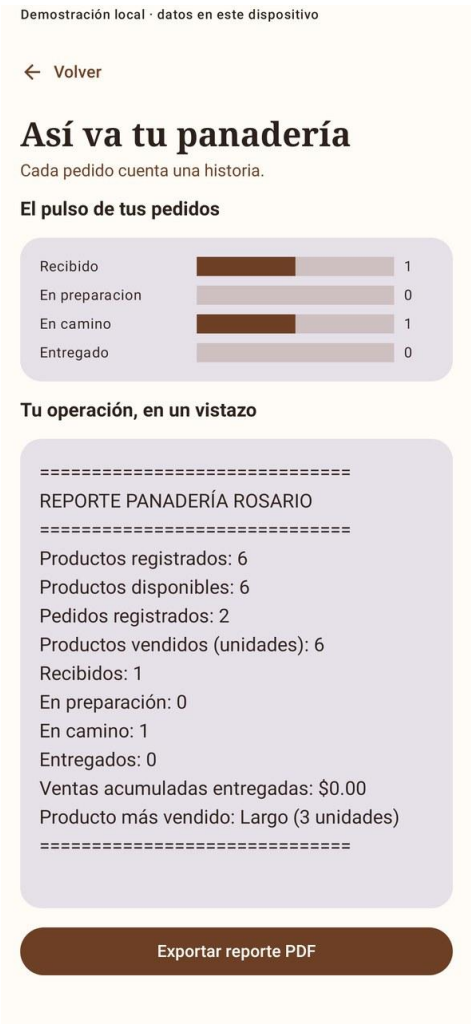
Nota. Captura de la interfaz de la aplicación en modo de demostración local, que utiliza las mismas pantallas que el modo conectado a Firebase. Elaboración propia.

Figura 5

Módulos de entregas y reportes



a) Pedidos en camino



b) Reporte y exportación PDF

Nota. Captura de la interfaz de la aplicación en modo de demostración local, que utiliza las mismas pantallas que el modo conectado a Firebase. Elaboración propia.

Correspondencia con los requerimientos del avance

Tabla 2

Requerimientos funcionales y su implementación

Requerimiento	Implementación en el proyecto
Autenticación con Firebase (correo y Google)	FirebaseRepository.login, register y google; pantallas LoginScreen y RegistroScreen
Persistencia en Firestore asociada al usuario	Colecciones usuarios y pedidos con el UID; reglas que aíslan los datos por usuario
Módulos secundarios funcionales	Catálogo, carrito, gestión de productos, pedidos, entregas y reportes PDF
Historial de registros almacenados	Pantalla Mis pedidos con listener filtrado por usuarioId; vista completa para el administrador

Nota. Elaboración propia.

Ventajas y limitaciones

Entre las ventajas destaca que Firebase elimina la necesidad de desarrollar y alojar un servidor de autenticación y una API para el catálogo y los pedidos. La gestión de contraseñas queda a cargo de un servicio especializado, los listeners ofrecen actualizaciones en tiempo real con poco código y las transacciones resuelven la concurrencia sobre las existencias. El plan gratuito resulta suficiente para un prototipo académico.

Entre las limitaciones, la lógica de negocio se ejecuta en el cliente, por lo que las reglas de seguridad son la única barrera frente a solicitudes manipuladas. Aunque las reglas del proyecto validan estados, propietarios y existencias, una operación comercial con cobros requeriría calcular los importes definitivos en un backend confiable, como Cloud Functions. También existe dependencia del proveedor: migrar los datos a otra plataforma exigiría reescribir la capa de acceso. Por último, las consultas de Firestore son más limitadas que SQL para reportes complejos, y el costo crece con el número de lecturas, lo que obliga a diseñar las consultas con cuidado.

Conclusiones

La integración de Firebase Authentication permitió reemplazar el acceso local por identidades verificables, con registro por correo, inicio de sesión con Google y sesiones

persistentes, sin almacenar contraseñas en la aplicación. El uso de custom claims separa de forma segura el rol de administrador del de cliente.

Cloud Firestore convirtió a Panadería Rosario en un sistema compartido: el catálogo, las existencias y los pedidos son los mismos en todos los dispositivos, el historial se asocia al UID de cada cliente y los cambios de estado se reflejan en tiempo real. Las transacciones y las reglas de seguridad garantizan la consistencia de los datos y el aislamiento entre usuarios.

Finalmente, la arquitectura basada en un repositorio permitió incorporar la nube conservando los modelos y servicios desarrollados en etapas anteriores, y el modo de demostración local facilita las pruebas mientras se configura el proyecto remoto. Como trabajo futuro se plantea trasladar la validación de importes a Cloud Functions e incorporar pagos electrónicos y recuperación de contraseña.

Referencias

- Android Developers. (s. f.). *Sign in your user with Credential Manager*., de https://developer.android.com/identity/sign-in/credential-manager
- Google. (s. f.-a). *Access data offline*. Firebase, de https://firebase.google.com/docs/firestore/manage-data/enable-offline
- Google. (s. f.-b). *Add Firebase to your Android project*. Firebase, de https://firebase.google.com/docs/android/setup
- Google. (s. f.-c). *Authenticate with Google on Android*. Firebase., de https://firebase.google.com/docs/auth/android/google-signin
- Google. (s. f.-d). *Cloud Firestore*. Firebase., de https://firebase.google.com/docs/firestore
- Google. (s. f.-e). *Cloud Firestore data model*. Firebase, de https://firebase.google.com/docs/firestore/data-model
- Google. (s. f.-f). *Control access with custom claims and security rules*. Firebase., de https://firebase.google.com/docs/auth/admin/custom-claims
- Google. (s. f.-g). *Firebase Authentication*. Firebase, de https://firebase.google.com/docs/auth
- Google. (s. f.-h). *Get realtime updates with Cloud Firestore*. Firebase, de https://firebase.google.com/docs/firestore/query-data/listen
- Google. (s. f.-i). *Get started with Cloud Firestore Security Rules*. Firebase, de https://firebase.google.com/docs/firestore/security/get-started
- Google. (s. f.-j). *Introduction to Firebase Local Emulator Suite*. Firebase, de https://firebase.google.com/docs/emulator-suite
- Google. (s. f.-k). *Transactions and batched writes*. Firebase., de https://firebase.google.com/docs/firestore/manage-data/transactions
- Moroney, L. (2017). *The definitive guide to Firebase: Build Android apps on Google's mobile platform*. Apress. https://doi.org/10.1007/978-1-4842-2943-9